

Aufgabe 3: Cache-Verhalten

```
.data
offset: .word 64
size: .word 8
array: .byte 1, 2, 3, 4, 5, 6, 7, 8

.text
la t0 array    # Beginn des Arrays
lw t1 size     # Größe des Arrays
add t1 t1 t0    # Ende des Arrays
lw t2 offset   # Offset beim Kopieren
add t2 t2 t0    # Beginn des kopierten Arrays

loop:
    bge t0 t1 end
    lb t3 0(t0)
    sb t3 0(t2)
    addi t0 t0 1
    addi t2 t2 1
    j loop
end:
    nop
```

3.1. Mögliche Cache-Konfigurationen (32 Words)

Die Gesamtgröße des Caches beträgt 32 Words. In allen Fällen werden die Strategien Write-back und Write-allocate verwendet.

Konfiguration 1:

- 32 Lines
- 1 Word pro Line
- Direct-mapped

Diese Konfiguration nutzt hauptsächlich temporale Lokalität. Räumliche Lokalität wird kaum ausgenutzt. Konflikt-Misses treten häufig auf.

Konfiguration 2:

- 8 Lines
- 4 Words pro Line
- Direct-mapped

Diese Konfiguration stellt einen guten Kompromiss dar. Sowohl temporale als auch räumliche Lokalität werden genutzt. Die Anzahl der Konflikt-Misses ist moderat.

Konfiguration 3:

- 2 Lines
- 16 Words pro Line
- Direct-mapped

Diese Konfiguration nutzt räumliche Lokalität sehr gut. Allerdings ist die Gefahr von Konflikt-Misses hoch, da nur wenige Cache-Lines vorhanden sind.

—

3.2. Wann ist Write-allocate nicht sinnvoll?

Write-allocate ist nicht sinnvoll, wenn Daten nur geschrieben, aber nicht erneut gelesen werden.

In solchen Fällen führt Write-allocate dazu, dass unnötige Cache-Lines aus dem Hauptspeicher geladen werden.

Typische Beispiele sind:

- Logging
- Streaming-Ausgaben
- Initialisierung großer Speicherbereiche

Hier ist No Write-allocate effizienter.

—

Um Programmieren die Wahl zwischen den beiden Strategien zu überlassen, könnte man die *store* Instruktion in *store with write around* und *store with write allocate* verzweigen:

```
# Syntax ist analog zu bestehenden RISC-V `store` Instruktionen
salw t0 0 t1 # (s)to(re) (w)ord using write (a)llocate
sarw t0 0 t1 # (s)to(re) (w)ord using write (a)r(ou)nd
```

Standard C hat keinen Mechanismus, um eine per-store cache-policy auszudrücken. Denkt man sich entsprechende Befehle aus, so trifft man auf ein Problem:

```
int foo;

// Ein neuer Befehl fuer jede Policy:
// denkbar, aber man kommt in Schwierigkeiten, wenn man die Funktionssignatur
// definieren will.
//
// Versuch mit einem generic <T>type (existiert nicht in C!):
setal((int) &foo, 1); // int setal(<T> *dest, <T> val): write value to adress
// specified by argument `dest` using write-miss policy write allocate

setar((int) &foo, 2); // int setar(<T> *dest, <T> val): write value to adress
// specified by argument `dest` using write-miss policy write around

foo = 3; // standard syntax: let compiler decide

// Problem: Es laesst sich in C nicht leicht ausdruecken, dass der Typ von `val`
// derselbe wie der des Pointers `dest` ist.
// Wir braechten soetwas wie generic Types.
// Oder eine gesonderte Funktion fuer jeden Datentypen: `setal_int, setal_float,
// setal_char`, usw.
// Die Reibung zeigt, dass diese Modifikation problematisch waere.
```

Es ist auch aus einer weiteren Perspektive fraglich, diesen Mechanismus in dieser Weise in C-Code zu exponieren – es wird hier eine Abstraktionsgrenze überschritten. Angebracht könnte es sein, entsprechende Compiler-Flags zu benutzen. Diese würden dann aber für das ganze translation unit gelten.

Fazit: einem C-Programm die Kontrolle über cache-policy write allocate vs. write around zu geben, ist einfacher gesagt als getan. Global liesse sich das per Compile-Flags erreichen. Aber um dies per store Anweisung zu kontrollieren, wären erhebliche Änderungen nötig (z.B. compiler extensions, oder intrinsics).

3.3. Vergleich der Write-Strategien bei Offset 64 und 128

Der verwendete Cache ist:

- Direct-mapped

- 8 Lines
- 4 Words pro Line

Der verwendete Datencache ist direct-mapped und besteht aus 8 Cache-Lines mit jeweils 4 Words pro Line.

Bei einem Offset von 64 Byte liegen die Quell- und Zieladressen in unterschiedlichen Cache-Lines. Daher treten nur wenige Konflikt-Misses auf.

Bei Verwendung von Write-back und Write-allocate ist die Hit-Rate in diesem Fall gut, da sowohl Lese- als auch Schreibzugriffe von der räumlichen Lokalität profitieren.

Bei Write-through und Write-allocate bleibt die Hit-Rate ebenfalls akzeptabel, jedoch verursacht jeder Schreibzugriff einen zusätzlichen Zugriff auf den Hauptspeicher.

Bei Write-back und No Write-allocate werden Schreibzugriffe nicht im Cache gespeichert. Die Lesezugriffe auf die Quelldaten profitieren weiterhin vom Cache, sodass die Hit-Rate insgesamt mittel ist.

Bei Write-through und No Write-allocate führen alle Schreibzugriffe direkt zu Speicherzugriffen. Dies verschlechtert die Performance, auch wenn keine starken Konflikt-Misses auftreten.

—

Bei einem Offset von 128 Byte entspricht der Abstand zwischen Quell- und Zieladressen der Gesamtgröße des Caches. In einem direct-mapped Cache werden beide Adressen auf dieselbe Cache-Line abgebildet.

Bei Write-back und Write-allocate führt dies dazu, dass sich Quell- und Zieldaten bei nahezu jedem Zugriff gegenseitig aus dem Cache verdrängen. Die Hit-Rate ist daher sehr schlecht.

Bei Write-through und Write-allocate tritt die gleiche Konfliktproblematik auf. Zusätzlich verursachen die Schreibzugriffe einen hohen Speicherverkehr.

Bei Write-back und No Write-allocate werden Zieladressen bei einem Write-Miss nicht in den Cache geladen. Dadurch bleiben die Quelldaten im Cache erhalten, und die Lesezugriffe haben eine gute Hit-Rate.

Auch bei Write-through und No Write-allocate bleiben die Quelldaten im Cache, da Schreibzugriffe den Cache umgehen. In diesem Fall ist die Hit-Rate ebenfalls gut.

Messergebnisse aus der Simulation (Ripes)

Die folgenden Messergebnisse wurden mit dem Simulator Ripes ermittelt. Es wurde ein gleitender Mittelwert über 50 Zyklen verwendet. Die Cache-Konfiguration entspricht der Aufgabenstellung.

—

Offset = 64 Byte

64	Hits	Misses
WB + WA	16	2
WT + WA	16	2
WB + NoWA	9	9
WT + NoWA	9	9

Table 1: Messergebnisse (Offset 64 Byte)

—

Offset = 128 Byte

128	Hits	Misses
WB + WA	2	16
WT + WA	2	16
WB + NoWA	9	9
WT + NoWA	9	9

Table 2: Messergebnisse (Offset 128 Byte)

Die Messergebnisse bestätigen die theoretische Analyse. Bei Offset 128 führen Write-allocate-Strategien zu starken Konflikt-Misses, da Quell- und Zieladressen auf dieselbe Cache-Line abgebildet werden. No Write-allocate verhindert diese Verdrängung und verbessert die Hit-Rate deutlich.

3.4 offset 128 und write allocate

Bei Offset 128 entspricht der Abstand zwischen Quell- und Zieladresse genau der Größe des gesamten Caches.

In einem direct-mapped Cache führt dies dazu, dass beide Adressen auf dieselbe Cache-Line abgebildet werden.

Bei Verwendung von Write-allocate verdrängen sich Quell- und Zieldaten gegenseitig aus dem Cache. Dies führt zu einer sehr niedrigen Hit-Rate.

—

Mathematischer:

Bei $\text{offset} = 128$ werden das `src` und `dest` auf denselben Index gemapped. Für einen Cache mit Assoziativitätsgrad 1 bedeutet das, dass sie auf denselben Index gemapped werden und um dieselbe Line konkurrieren. In der Schleife wird `src` per `load` gecached und dann gleich wieder von `dest` in der `store` Instruktion verdrängt. Das wiederholt sich und wir sehen entsprechende Verschlechterung in der Hit-Rate.

Begründung: Wir benutzen die Formel

$$\text{index} = (\text{address} \gg \text{offset_bits}) \bmod \text{number_of_lines}$$

Für uns ergibt das:

$$\text{index} = (\text{address} \gg 4) \bmod 2^3$$

Index wird also bestimmt durch Bits 4, 5, 6 der Adresse.

Untersuchen wir, was genau bei $\text{address} + 64$ bzw. $\text{address} + 128$, passiert sehen wir, dass wegen $64 = 2^6$; $128 = 2^7$ in dem einen Fall die Relevanten Bits veraendert werden, im anderen allerdings nicht. Resultat: die Speicherbloecke werden einmal verschiedenen Indizes zugeordnet, und einmal demselben.

Man koennte das Problem umgehen, indem man die Assoziativitaet auf 2 anhebt.– Dadurch koennten src und dest im selben Set gecached werden. Moegliche Konfiguration: 4 Sets a 2 Bloecke a 4 Words (Assoziativitaetsgrad 2).

3.5 Cache-Hits maximieren

Die zuendende Idee ist, loads und stores nicht zu verflechten, sondern zu batchen: wir verwenden die ganze geladene Cache Line, erst danach gehen wir zur write-Phase ueber, in der die Line evtl. verdraengt wird. Verdraengung skaliert nun mit $n/\text{size cache line}$ und nicht mehr mit n .

Im Falle unseres Arrays, muessten wir einfach den loop ersetzen:

```
lw t3 0(t0)
lw t4 4(t0)
sw t3 0(t2)
sw t3 4(t2)
```

und kommen damit auf eine Hit-Rate von 67%.

Fuer eine skalierte Version bietet sich eine Schleife an, die mit ganzen Cache-Lines arbeitet:

```
// N = array length in words
// step of 4 -> our cache block size is 4 words
for i in 0..N-1 step 4:
    w0 = load word src[i+0]
    w1 = load word src[i+1]
    w2 = load word src[i+2]
    w3 = load word src[i+3]

    store word dest[i+0] = w0
    store word dest[i+1] = w1
    store word dest[i+2] = w2
    store word dest[i+3] = w3
```